Estimation of the background (example3.py)

1) The implementation of the loop that generates **frameIds** vector elements is the while loop that counts from 0 to 100, adding each number to the list. It selects the first 101 frames of the video.

2) Different methods for determining the mean and median in the image vector numpy.mean and numpy.median object functions:

   - np.mean adds all pixel values together and divides by 101

   - np.median sorts all pixel values and picks the middle one

3) Difference between the mean and median results, complexity and computation time of both algorithms:



Current frame



Mean frame



Median frame

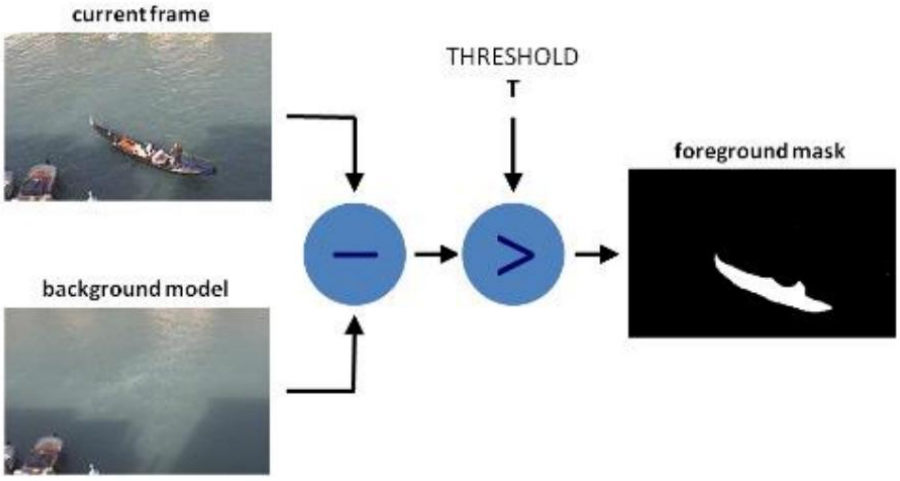
My observations:

- Both methods successfully remove most cars from the background
- In this specific case with 101 frames, mean produced a slightly cleaner result (because we see less traces from where cars used to be)
- The median still shows faint car traces because the traffic was dense — some cars occupied the same lane position frequently enough to influence the median value
- Mean is faster (O(n)) but more sensitive to outliers
- Median is slower (O(n log n)) but theoretically more robust — it seems like it needs more frames to work well on busy roads

Foreground mask generation (example4.py)

Theory: Background subtraction (BS) is a common and widely used technique for generating a foreground mask (namely, a binary image containing the pixels belonging to moving objects in the scene) by using static cameras.

As the name suggests, BS calculates the foreground mask performing a subtraction between the current frame and a background model, containing the static part of the scene or, more in general, everything that can be considered as background given the characteristics of the observed scene.



Background modeling consists of two main steps:

1. Background Initialization;
2. Background Update.

In the first step, an initial model of the background is computed, while in the second step that model is updated in order to adapt to possible changes in the scene.

After running example4.py these are the result images:



Current frame



Backgroun model



Difference treshold

Note the absdiff and threshold functions:

cv2.absdiff

- Takes two images and calculates the absolute difference between each pixel
- E.g. if background pixel = 100 and current frame pixel = 140, the result is 40
- Moving cars produce high difference values, static background produces values near 0

cv2.threshold

- Takes the difference image and converts it to pure black and white
- Pixels above treshold becomes white (255, moving object)
- Pixels below treshold become black (0, background)
- Any pixel difference below 25 → becomes black (0) = background

The process:

currentFrame - medianBackground = difference → threshold → black/white mask

White blobs in the result = cars moving on the highway

Example – Background subtractor – Mixture of Gaussians, KNN (example5.py)

history

- how many past frames are used to build the background model
- more frames = more stable but slower to adapt

varThreshold

- Higher = less sensitive, fewer detections but less noise
- Lower = more sensitive but noisier mask

detectShadows

- True = shadows shown in grey instead of white, more accurate but slower

	MOG2	**KNN**
How it works?	Models each pixel as a mixture of Gaussians	Uses K-nearest neighbours to classify pixels
Speed	Faster	Slower
Best for	General use	Scenes with lots of colour variation

Image 1 — MOG2 default

1. Cars are detected well with clear white blobs
2. **Lots of noise** in the trees and grass areas on the sides
3. Shadows visible merged together with the car shapes

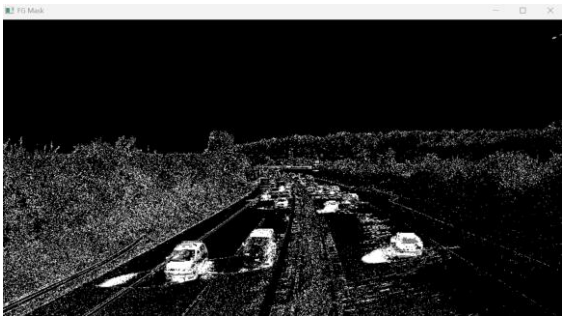


Image 2 — MOG2 history=120, varThreshold=50, detectShadows=True

1. **Cleanest result** — much less noise on the sides
2. Cars are clearly separated as distinct white blobs
3. Higher threshold (50) means only significant differences are detected
4. This is the best performing setting for this video



Image 3 — MOG2 history=50, varThreshold=25, detectShadows=False

1. **Most noise** — the trees and sides are very white and speckled
2. Cars are harder to distinguish from the background noise
3. Low threshold (25) picks up too many small differences
4. Short history (50) means the model adapts too quickly

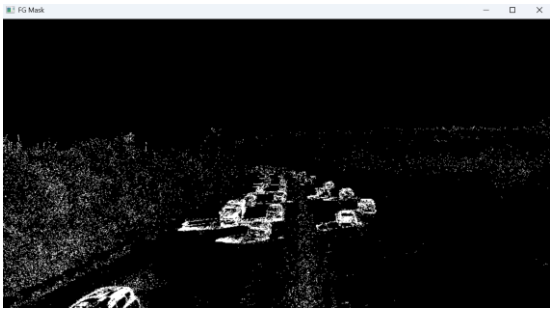


Image 4 — KNN

1. Similar to MOG2 default but cars appear as **bigger, chunkier blobs**
2. Still quite noisy on the sides
3. Car shapes are less precise than MOG2



Key conclusion:

MOG2 with varThreshold=50 gave the best results for this highway video. It has cleanest separation between cars and background with minimal noise. Lower threshold and shorter history made detection worse in this case.

Task

Mean background model

- Much **more noise** across the entire image — trees, road edges all appear white
- The entire road area is almost completely white, making it hard to distinguish individual cars
- Mean is sensitive to lighting changes and outliers, so more false detections appear

Mean:



Median background model

- **Much cleaner** result — individual cars are clearly visible as separate white blobs
- Less noise in the tree areas compared to mean
- Cars on both lanes are clearly distinguishable

Median:



Key conclusion:

The median background method is better than the mean because it is more robust to moving cars and outlier pixel values. However, compared with MOG2/KNN, MOG2 gives the best overall results because it produces cleaner car blobs and continuously adapts to background changes. The median method is simpler and works well when the camera and lighting are stable, but MOG2 is more reliable for real-world video conditions.

Best accuracy/reliability: MOG2
Simplest method: Median background
Worst of these for this task: Mean background